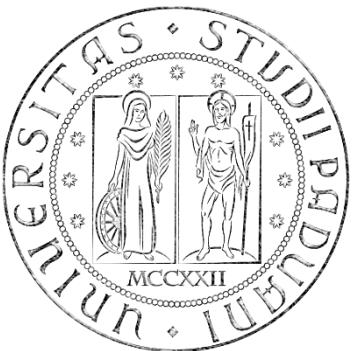


Biomedical Wearable Technologies for Healthcare and Wellbeing

Introduction

A.Y. 2025-2026
Giacomo Cappon



Who am I

➤ Giacomo Cappon

- Assistant Professor, Department of Information Engineering, University of Padova
- PhD in Information Engineering, curriculum Bioengineering
- Main research interests: signal processing; decision support systems; wearable sensors; digital health; ai for medicine



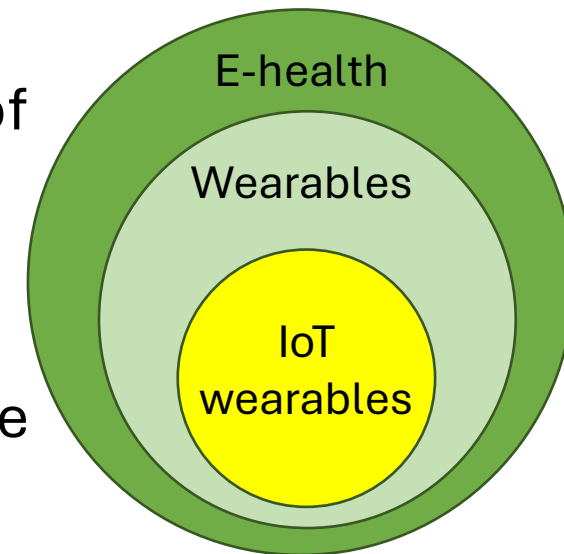
➤ Contacts

- email: giacomo.cappon@unipd.it
- personal website: <https://gcappon.github.io>
- public repos: <https://github.com/gcappon>

What is the course about?

Biomedical wearable technologies...

- **E-health or digital health:** The use of modern Information and Communication Technologies (ICT) to efficiently deliver all kinds of healthcare services
 - Examples: EHR, telemedicine systems
- **Wearables:** electronic devices worn close to and/or on the surface of the skin, which detect, analyze and transmit biomedical data, such as heart rate, activity, blood pressure, temperature, etc.
- **Internet-of-thing (IoT) wearables:** Wearables that connect and exchange data with other devices (wearable or not) and systems over the Internet or other communication networks



What is the course about?

... for healthcare and wellbeing

- Our society is facing some important global health challenges
 - **Lifestyle-related illnesses** driven by sedentary life, consumption of processed and fast foods, alcohol drinking and smoking
 - Need to change the paradigm of healthcare, from a mostly reactive system to a **proactive system**
 - **Population ageing** rises the demand of health services and the healthcare expenditure.
 - Need to **decentralizing the healthcare**
 - Covid-19 pandemic called for urgent delivery of healthcare at home

Biomedical wearable technologies are fundamental to make healthcare scalable and accessible outside the hospital setting and deliver personalized and preventive health services

Theory program

- The theoretical part of the course will cover the following:
 - Wearable sensor technologies
 - Health design thinking
 - Elements of UI/UX Design
 - Evaluation of app usability
 - Fundamentals of network architectures and the HTTP communication protocol
 - The representational state transfer (REST) paradigm
 - Application programming interfaces (APIs)
 - Elements of network security
 - Authentication protocols
 - Regulation for health data management

Lab program

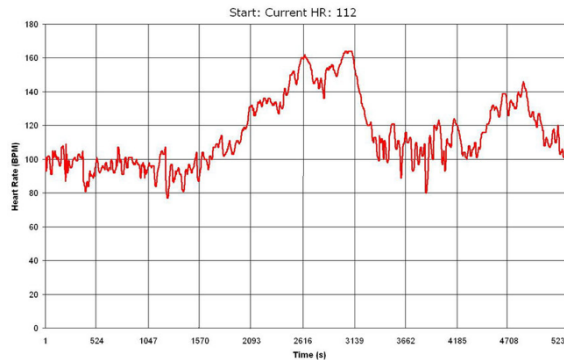
➤ The laboratorial part of the course will cover the following:

- The Git version control system
- Dart programming language and the Flutter framework
- Building mobile/web applications in Flutter
- User Interface design
- Navigation
- State management
- Basic data persistence
- Networking
- Integrating AI models in mobile applications

➤ Workshop...

Lab program - The workshop

- 2-hours workshop where we will set up a fake clinical trial aimed at evaluating the cardiovascular tone in a cohort of healthy subjects using an experimental algorithm



- The fastest 5 students to submit the right answer will get a +1 to the final evaluation.

1. Trial setup

2. Data collection using a wearable and an app

3. Data preparation

Together

4. Data analysis

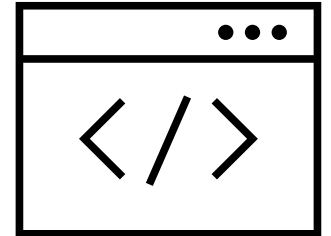
5. Trial outcome

Individually

Prerequisites & End of class goals

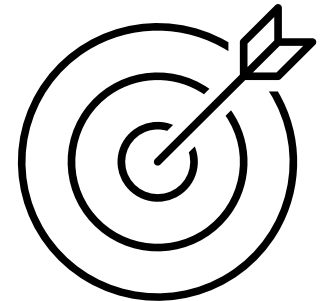
➤ Prerequisites

- Basic:
 - Propension for programming, Python
- Preferable:
 - Object-oriented programming



➤ End of class goals

- Knowing how wearable devices communicates to the end-user
- Designing digital solutions to enhance healthcare and well-being
- Developing a fully functioning mobile application



The exam

➤ **Written test:** Theory exam

- 4 open questions
- Duration: 90 min
- Max vote: 20/30

Dates of the written test

- First attempt: 18 Jun 2026, 14:00 (Room Me+Ne)
- Second round: 07 Jul 2026, 14:00 (Room De)
- Third round: 28 Aug 2026, 14:00 (Room De)
- Fourth round: 01 Feb 2027, 14:00 (Room Ee)

➤ **Project:** Development of an application that integrates and elaborates data from a smartwatch. To be done in groups of 3 students.

- Duration of the discussion: 45 min (25 min presentation + 20 min Q&A)
- Max vote: 12/30

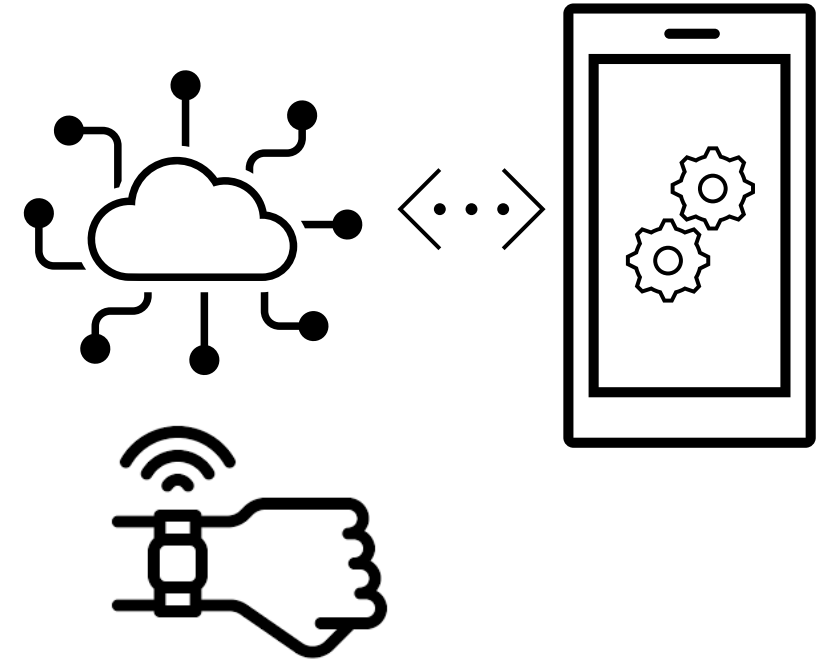
➤ **Final mark:** sum of the votes of the written test and the project.

Project: What's the spirit here

- The goal is learning “**How to do SOMETHING ORIGINAL using wearable devices**”
- What **I DO** expect from you
 - To use your imagination to create that SOMETHING
 - To learn to build something from scratch
 - To use code built by others
 - To work together
 - To use Google and StackOverflow
- What **I DO NOT** expect from you
 - To create the new Google
 - To reinvent the wheel (copying others code is fine)
 - To create extra complicated UI with animations and stuff

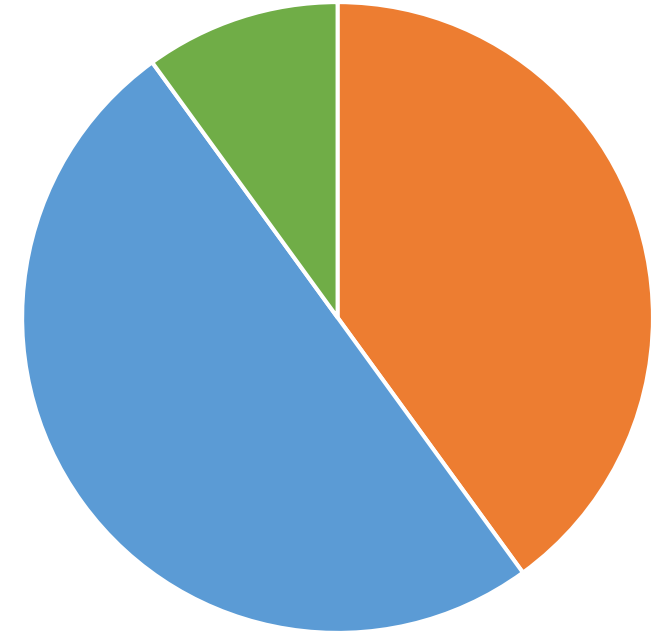
Project structure

- The project consists of building an app for iOS or Android that collects user data from a wearable device through Web APIs, visualizes them, and does some tricks with them.
- Core functionalities:
 - Correct app flow
 - User authentication and management
 - Data collection
 - Data visualization and presentation
- Additional functionalities → It's up to you!
Some examples:
 - Run some analysis on data and provide suggestions to the user
 - Implement some literature algorithm
 - ...



Grading

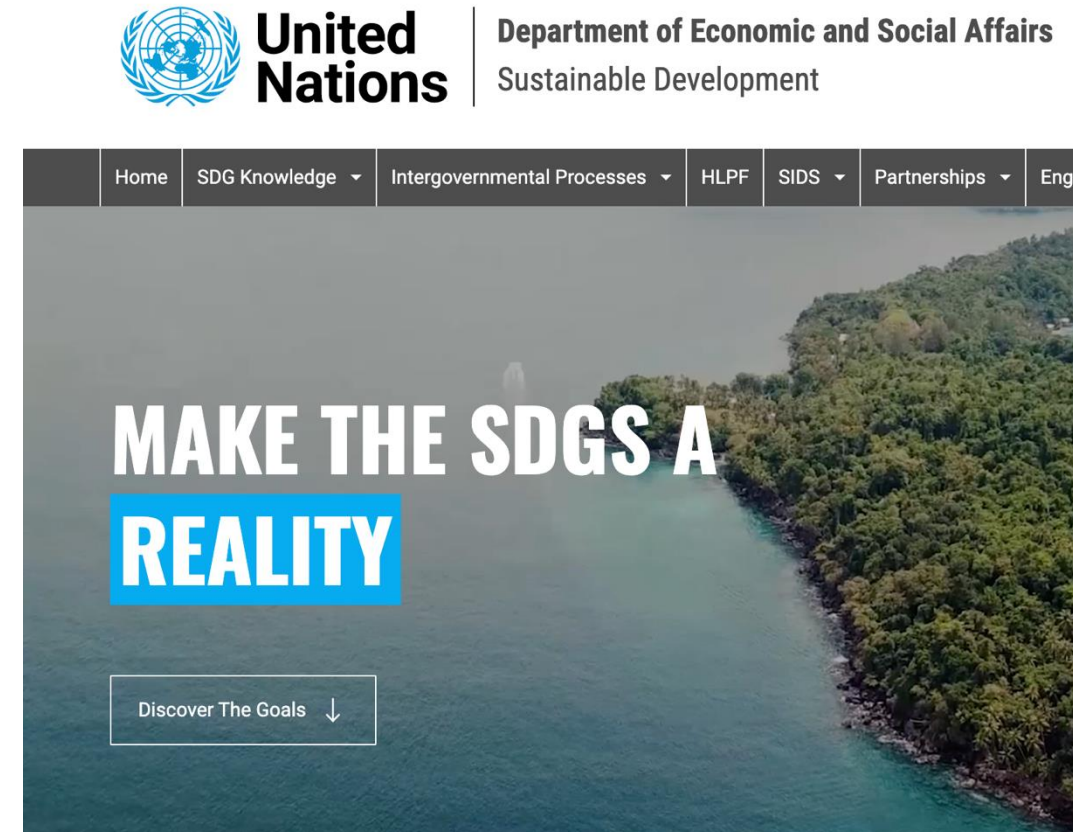
- What are the grading criteria
 - Compliance to core functionalities
 - Originality of the additional feature
 - Quality of teamwork
- How much these criteria matter?
 - Core functionalities: 40%
 - Additional app functionalities: 50%
 - Quality of teamwork: 10%



- Core functionalities
- Additional functionalities
- Quality of teamwork

Choose your target

- Your app must represent a solution for the current SDGs (Sustainable Development Goals) targets
- SDG are 17 topics representing the current big challenges humanity must face and solve



Choose your target

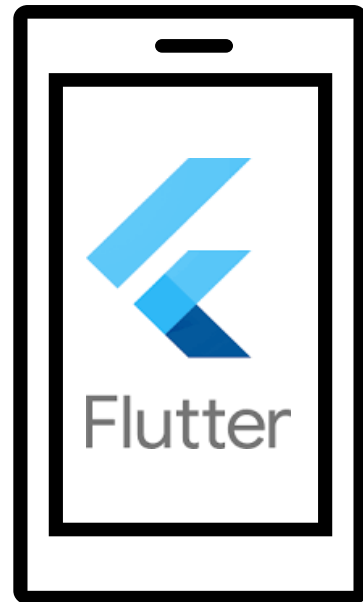
- These are the target you can choose from (see the elearning page for more details):
- **SDG 2: Target 2.2** (By 2030, end all forms of malnutrition, including achieving, by 2025, the internationally agreed targets on stunting and wasting in children under 5 years of age, and address the nutritional needs of adolescent girls, pregnant and lactating women and older persons)
 - **SDG 2: Target 2.3** (By 2030, double the agricultural productivity and incomes of small-scale food producers, in particular women, indigenous peoples, family farmers, pastoralists and fishers, including through secure and equal access to land, other productive resources and inputs, knowledge, financial services, markets and opportunities for value addition and non-farm employment)
 - **SDG 3: Target 3.4** (By 2030, reduce by one third premature mortality from non-communicable diseases through prevention and treatment and promote mental health and well-being)
 - **SDG 3: Target 3.5** (Strengthen the prevention and treatment of substance abuse, including narcotic drug abuse and harmful use of alcohol)
 - **SDG 4: Target 4.7** (By 2030, ensure that all learners acquire the knowledge and skills needed to promote sustainable development, including, among others, through education for sustainable development and sustainable lifestyles, human rights, gender equality, promotion of a culture of peace and non-violence, global citizenship and appreciation of cultural diversity and of culture's contribution to sustainable development)
 - **SDG 8: Target 8.9** (By 2030, devise and implement policies to promote sustainable tourism that creates jobs and promotes local culture and products)

Data source

- Data that you will use are collected by me using a Fitbit Versa 2
- Data are available starting from 09/02 and will be collected up to 30/11
- The following data “types” are included(see mor details in the Appendix):
 - Calories
 - Distance
 - Exercise sessions
 - Heart rate
 - Resting heart rate (Estimated)
 - Sleep
 - Steps
- Your project must use at least 1 type of data



How to get data - The IMPACT backend



Your app

HTTP
requests/responses



<https://impact.dei.unipd.it/bwthw/>

- You will be provided with credentials (different for each group) and learn how to get data during the lab lessons

Optional tutoring activities

- Need help? They are here to cover your back with answers and optional tutoring activities



➤ Luca Cossu

luca.cossu@phd.unipd.it



➤ Alberto Gastaldello

alberto.gastaldello.2@phd.unipd.it

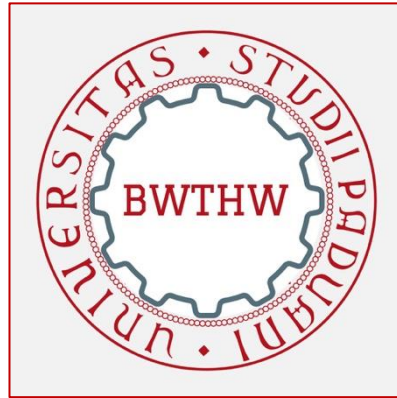
Calendar of the optional tutoring activities

Day	Time	Room
Monday, March 2, 2026	14:30 - 16:00	TBD
Monday, March 9, 2026	14:30 - 16:00	TBD
Monday, March 16, 2026	14:30 - 16:00	TBD
Monday, March 30, 2026	14:30 - 16:00	TBD
Monday, April 13, 2026	14:30 - 16:00	TBD
Monday, May 11, 2026	14:30 - 16:00	TBD
Monday, May 18, 2026	14:30 - 16:00	TBD
Monday, May 25, 2026	14:30 - 16:00	TBD
Monday, June 8, 2026	14:30 - 16:00	TBD

OR

Day	Time	Room
Tuesday, March 3, 2026	16:30 - 18:00	TBD
Monday, March 9, 2026	14:30 - 16:00	TBD
Tuesday, March 17, 2026	16:30 - 18:00	TBD
Tuesday, March 31, 2026	16:30 - 18:00	TBD
Tuesday, April 14, 2026	16:30 - 18:00	TBD
Tuesday, May 12, 2026	16:30 - 18:00	TBD
Tuesday, May 19, 2026	16:30 - 18:00	TBD
Tuesday, May 26, 2026	16:30 - 18:00	TBD
Tuesday, June 9, 2026	16:30 - 18:00	TBD

Discord server



+



➤ Join the Discord server to get in touch and ask questions

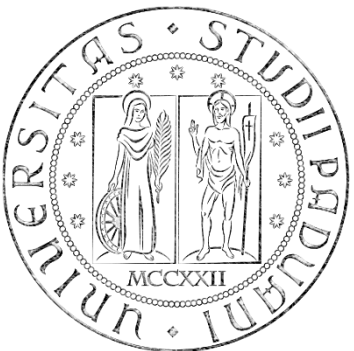
➤ Link: <https://discord.gg/j76A3D27bm>



Biomedical Wearable Technologies for Healthcare and Wellbeing

Wearables

A.Y. 2025-2026
Giacomo Cappon

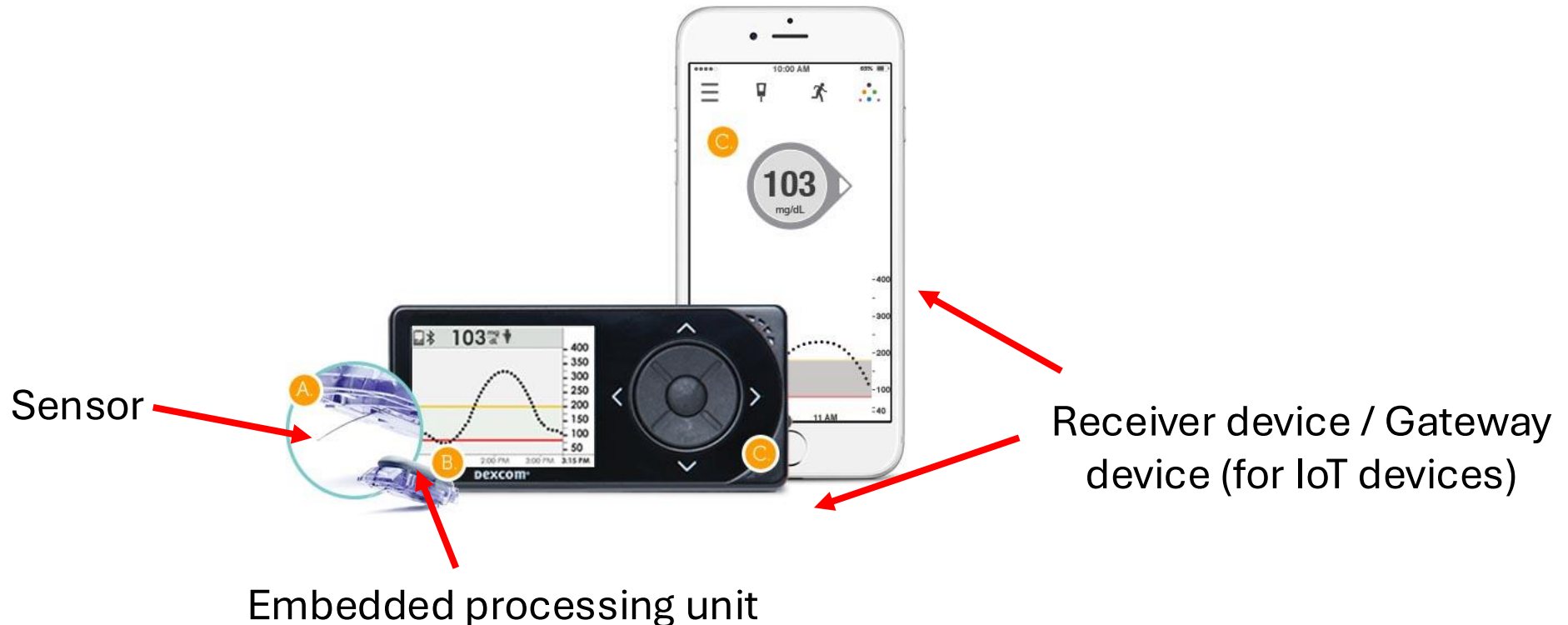


Wearable devices



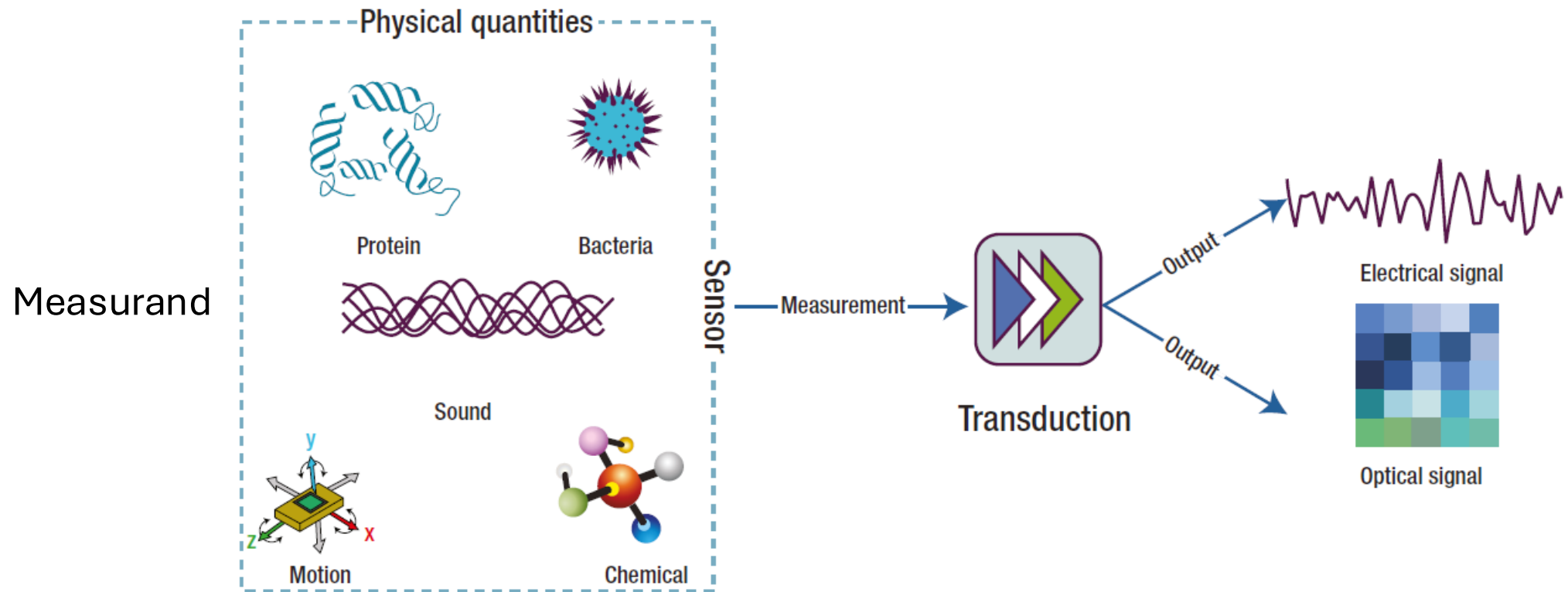
Wearable devices

- A wearable device is a device that can be used for sensing physiological parameters or signals which generally include two components:
- A **sensor** that measures the physical quantity of interest (measurand)
 - An **embedded processing unit** that does some initial processing of the measurements and either stores them in a memory or sends them to another device



Definitions of sensor

- “A sensor generally refers to a device that converts a physical measure into a signal that is read by an observer or by an instrument” (Chen et al., 2012)

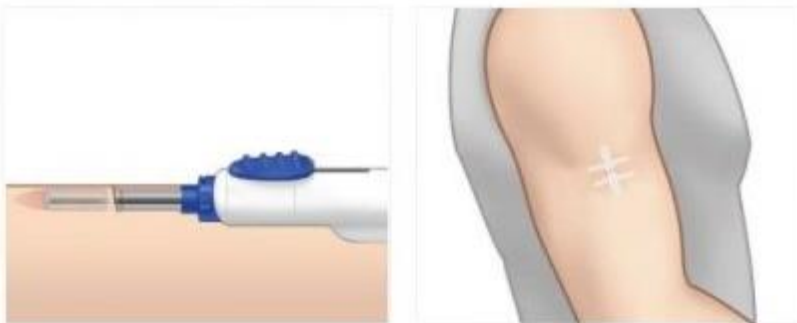


Classification of sensors based on their interaction with the measurand

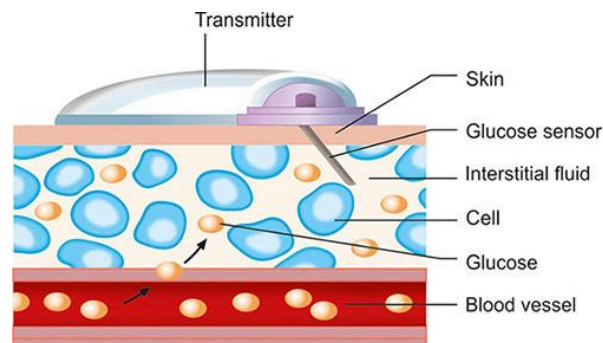
Contact sensor: sensors requiring physical contact with the measurand.

- Their deployment obviously perturbs the state of the subject to some degree.
 - Comfort and biocompatibility are important considerations
 - Response time determined by the speed at which the measurand is transported to the measurement site.
 - Poor contact can result in signal noise and the introduction of signal artifacts.

Invasive: sensors inserted into human body



Minimally-invasive: patch-type devices on the skin that monitor interstitial fluids



Non-invasive: sensors that simply have contact with the skin without effect



Classification of sensors based on their interaction with the measurand

Non-contact sensors: do not require direct contact with the measurand.

- Minimum perturbation of the subject
- Generally used in ambient sensing applications
- Example: infrared thermometer

Sample based sensors: require an invasive collection of a sample of the person to be monitored by a human or automated sampling system.

- The collected samples are analyzed using sensors or laboratory-based analytical instrumentation
- Common approach in healthcare (e.g. monitoring cholesterol)

Classification of sensors based on their sensing technique

- Mechanical sensors
- Electrical sensors
- Optical sensors
- Chemical sensors

Mechanical sensors

➤ **Mechanical sensors** detect some form of mechanical deformation of a material, in response to the measurand (input), and translate it to an electrical, optical, magnetic or thermal signal (output).

➤ **Electromechanical sensors:**

Measurand → Mechanical deformation → Electrical signal

➤ Types of electromechanical sensing techniques:

- Piezoresistive sensors
- Capacitive sensors
- Iontronic sensors
- Piezoelectric sensors

Electrical sensors

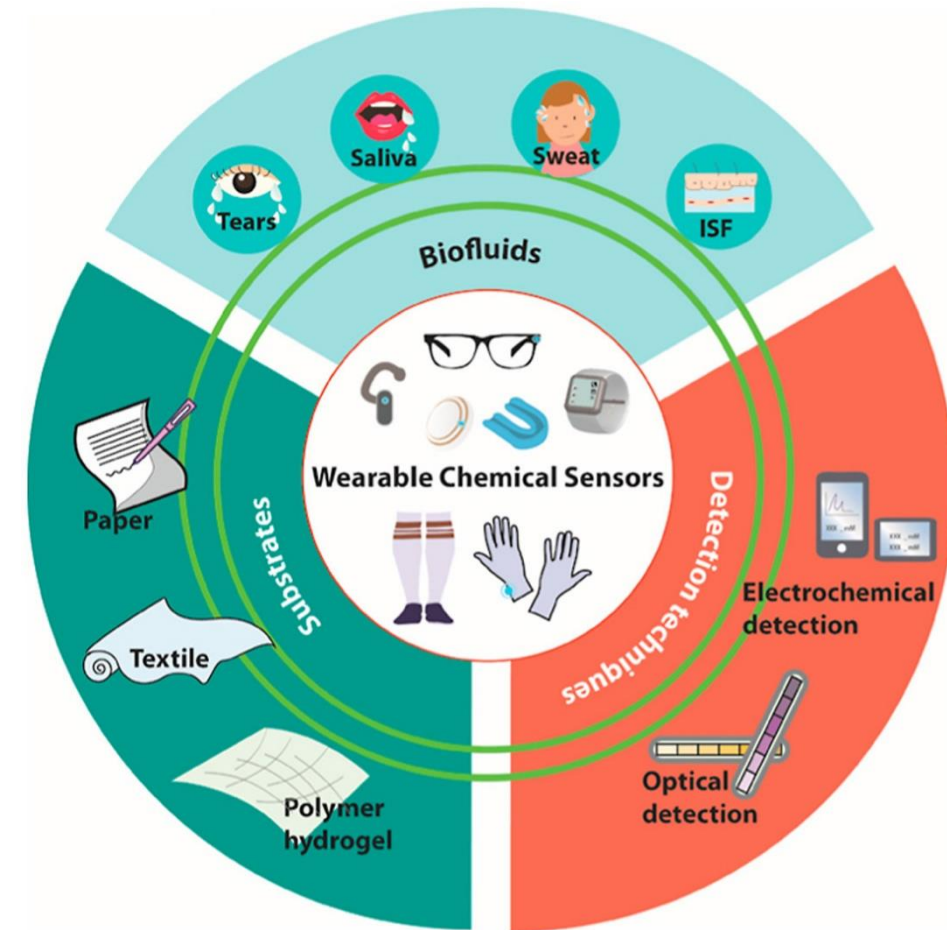
- **Electrical sensors:** on-skin electrodes that measure a change in electrical resistance or the electrical charge of the skin.
- Good electrical contact with skin is required.
- **Wet electrodes:** solid conductive pad interfaced to the skin via an electrolyte gel that minimizes the impedance of skin by hydrating it and forming a conformal electrical contact with its textured surface.
- **Dry electrodes:** direct contact with the skin, sensible to movement artifacts.
- Skin electrodes allow to collect high-fidelity measurements of a broad range of physiologically relevant biopotentials:
 - electrocardiogram (ECG)
 - electroencephalogram (EEG)
 - electromyogram (EMG)
 - electrooculogram (EOG)
 - electroretinogram (ERG)
 - galvanic skin response (GSR), also known as skin impedance or electrodermal activity (EDA)



Optical sensors

- **Electro-optical sensors** detect light and convert it to an electrical signal
 - *Photoconductive* devices convert a change of incident light into a change of resistance.
 - *Photovoltaics* (solar cells) convert an amount of incident light into an output voltage.
 - *Photodiodes* and *phototransistor* convert an amount of incident light into an output current.
- Operating principle of wearable optical sensors: a light source introduces light into the body and based on how much the light is absorbed or reflected, measurements related to the body tissue are derived.
- The wavelength of light sources can range from UV to deep infrared, depending on needed penetration depth and the absorption peak for the relevant sensing application.
- Commonly used for monitoring of heart rate and blood oxygenation.

Chemical sensors



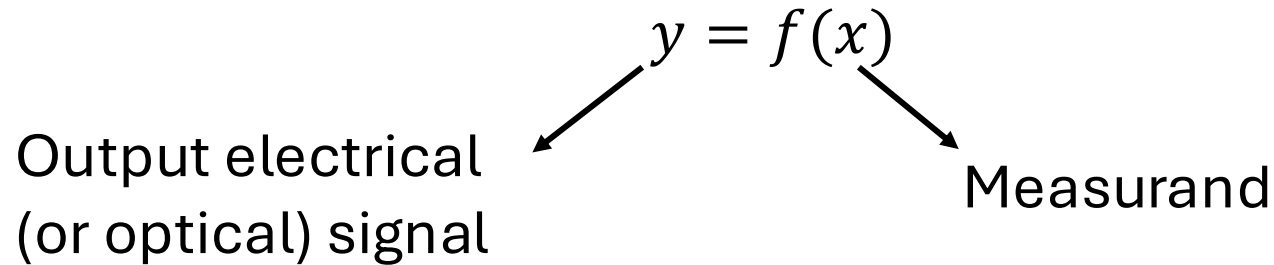
- They quantify chemical markers, or analytes, in biological fluids and convert this information in an optical (**optochemical sensor**) or electrical signal (**electrochemical sensor**).
- Require direct chemical interaction with the biomarker (contact sensors).
- Wearable chemical sensors use non-invasive or minimally-invasive techniques to measure analytes in **alternative biofluids** instead of invasive blood sampling.
 - Biofluids secreted by the body (saliva, sweat, tears)
 - Interstitial fluid reached through minimally-invasive sensors or by reverse iontophoresis

Sensor characteristics

- To truly understand sensors, and how sensors that measure the same measurand can differ, it is necessary to understand sensor performance characteristics, which are reported in **specifications** (specs) or **datasheet**.
 - Systematic characteristics: those which can be exactly quantified by mathematical or graphical means;
 - Statistical characteristics: those which cannot be exactly quantified;
 - Dynamic characteristics: those characterizing the ways in which an element responds to sudden input changes.

Transfer function

- A function f that describes the relationship between the measurand and the output signal (electrical or optical).

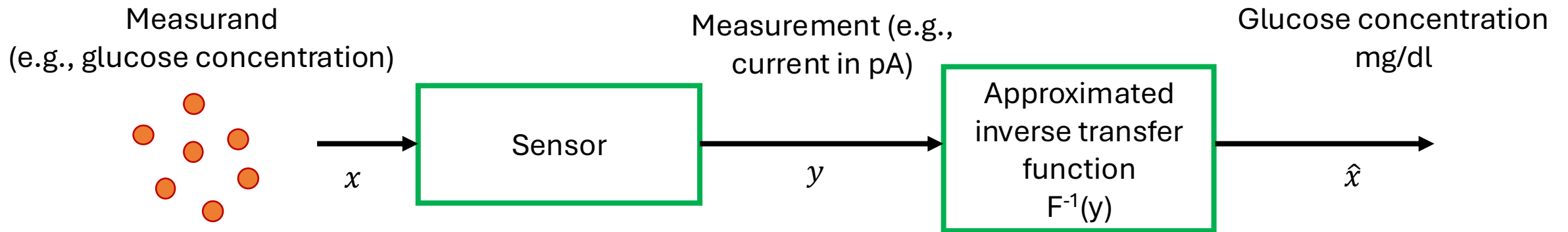


- Represented in specs as a table of values, a graph, or a mathematical formula.
- Typically the actual transfer function is complex → the transfer function is approximated by a simpler linear or non-linear function, $F(\cdot)$, which can be mathematically expressed as a formula.

$$y = F(x) + e \quad e = \text{approximation error}$$

Inverse transfer function

- The embedded processing unit of the sensor device applies the inverse of $F(\cdot)$, $F^{-1}(\cdot)$, to convert the electrical signal generated by the sensing technique to output measurements in the same domain of the measurand.



Common transfer function approximations

Linear transfer function approximation:

$$y = a \cdot x + b + e$$

- The offset, b , is the output value of the sensor when no measurand is applied. It is the component of the output signal which does not depend on the measurand.
- The slope, a , represents the **sensor sensitivity**, i.e., the change in the sensor output resulting from a unit change in the input measurand.

Non-linear transfer function approximations:

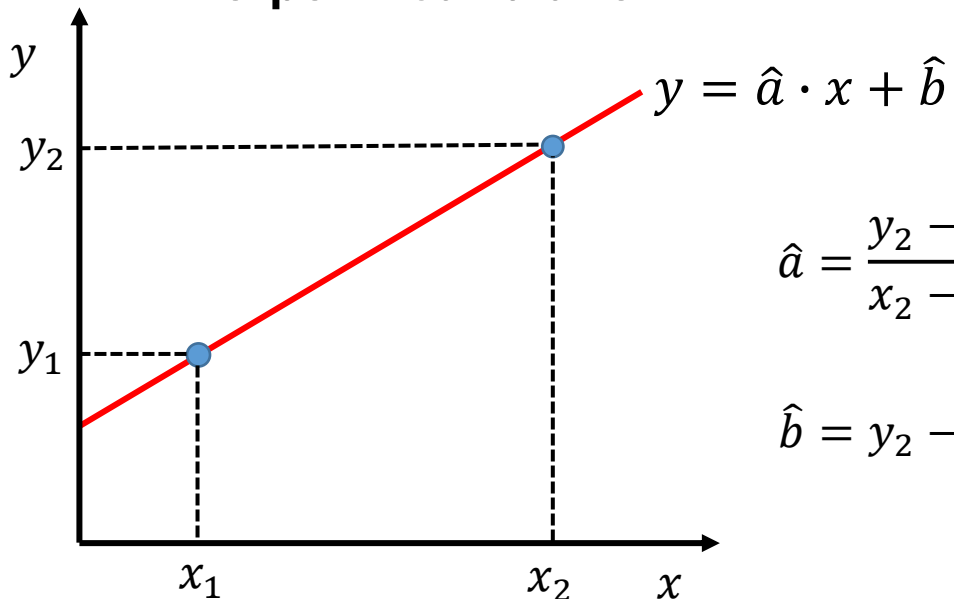
Logarithmic function	$y = a + b \cdot \ln(x) + e$
Exponential function	$y = a \cdot e^{k \cdot x} + e$
Power function	$y = a + b \cdot x^k + e$
Second-order polynomial function	$y = a \cdot x^2 + b \cdot x + c + e$
Third-order polynomial function	$y = a \cdot x^3 + b \cdot x^2 + c \cdot x + d + e$

Calibration

- The transfer function parameters, e.g., a and b in the linear transfer function, are in general unknown and must be estimated. This process is called **calibration**.
- If a set of known (x,y) paired values are available, estimation of transfer function parameters can be performed by the least square approach.
- For linear transfer function:

$$(\hat{a}, \hat{b}) = \operatorname{argmin}_{a,b} \sum_{i=1}^n [y_i - (a \cdot x_i + b)]^2$$

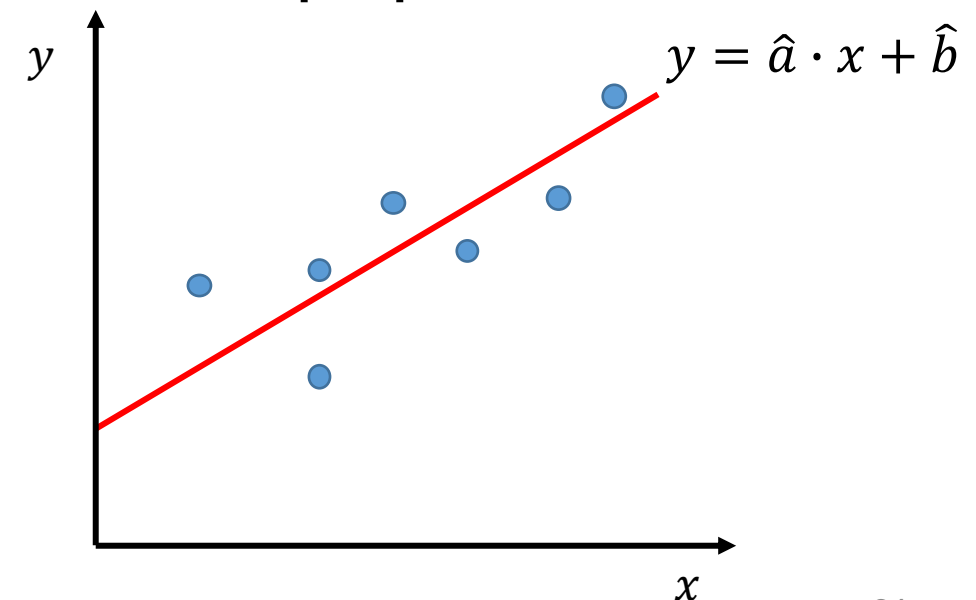
Two-point calibration



$$\hat{a} = \frac{y_2 - y_1}{x_2 - x_1}$$

$$\hat{b} = y_2 - \frac{y_2 - y_1}{x_2 - x_1} \cdot x_2$$

Multiple-point calibration



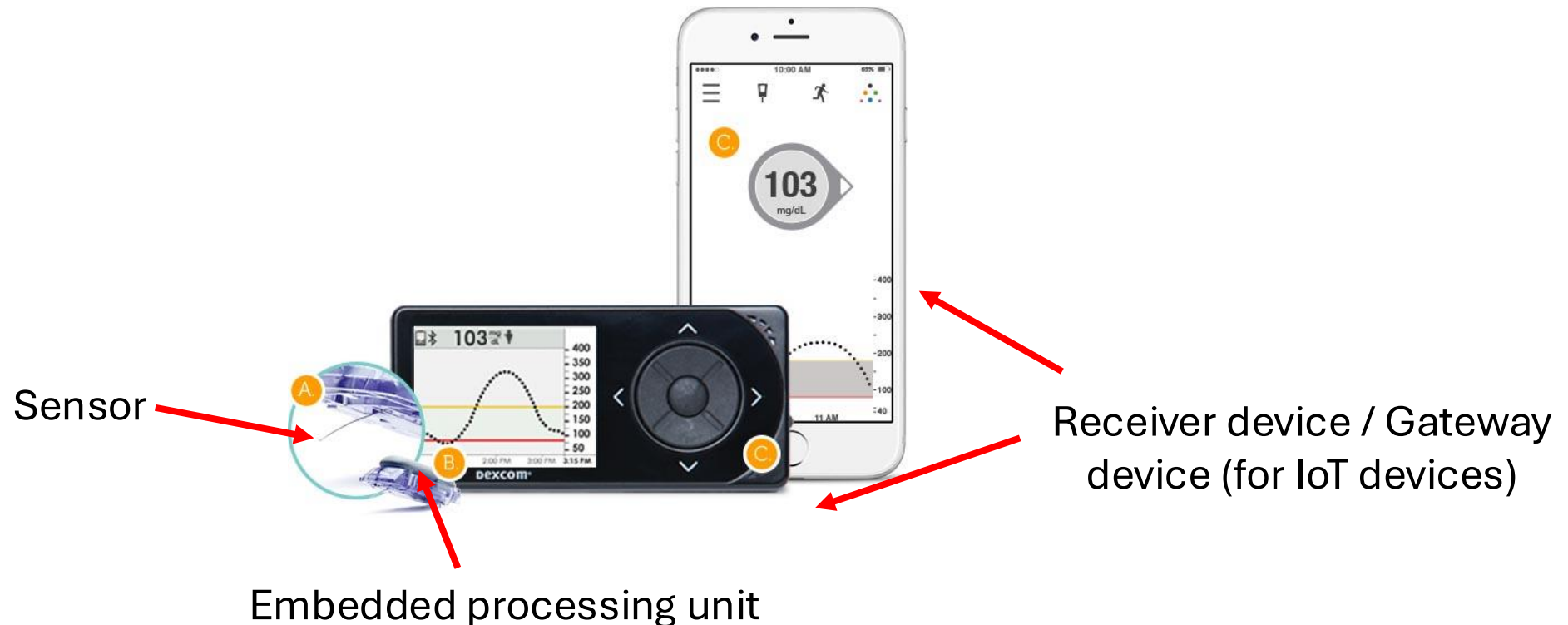
Embedded processing unit

➤ The embedded processing unit is responsible for:

- Performing local processing of the measurements
- Storing some data in a local memory
- Sending the data to another device (e.g., a display device or a gateway device for IoT systems)

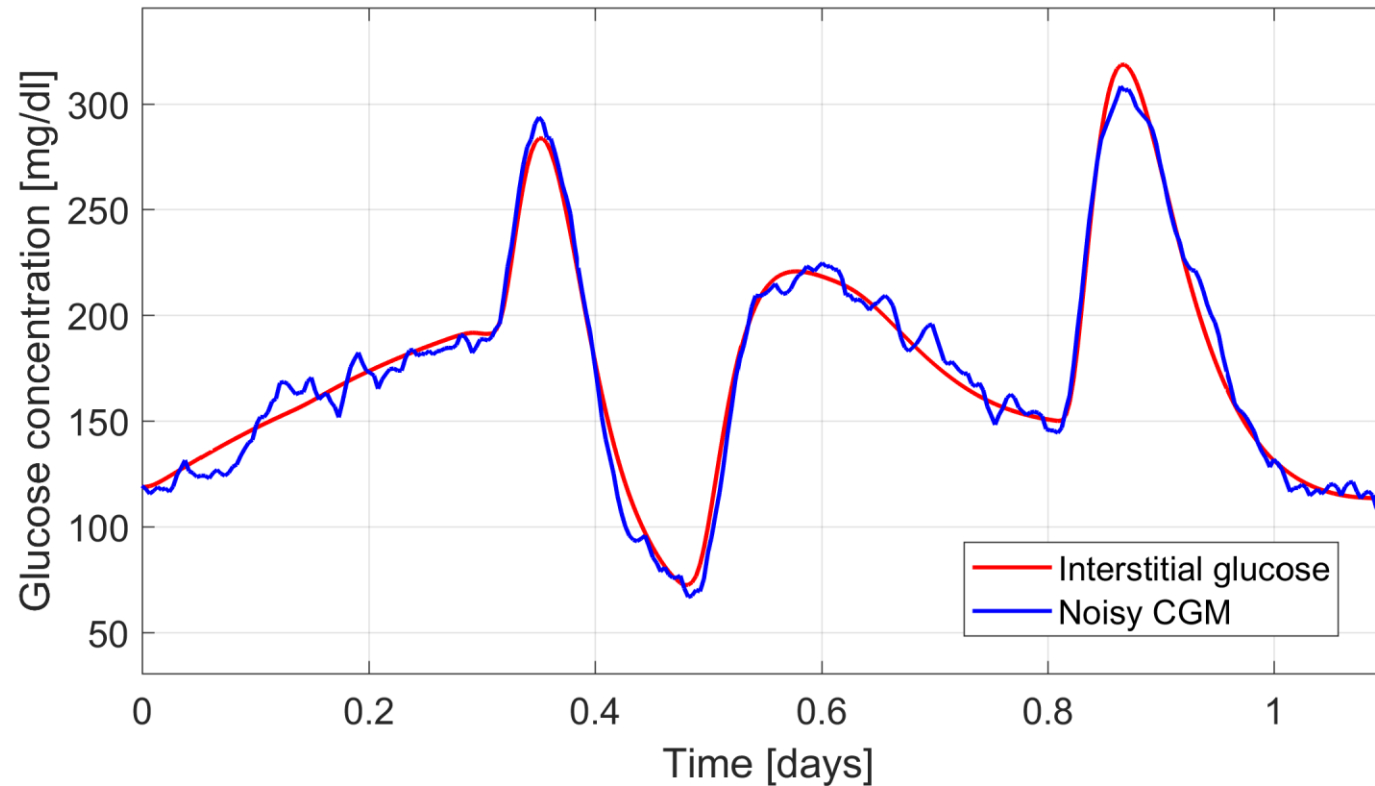
Three main operations:

- Denoising
- Calibration
- Fault detection



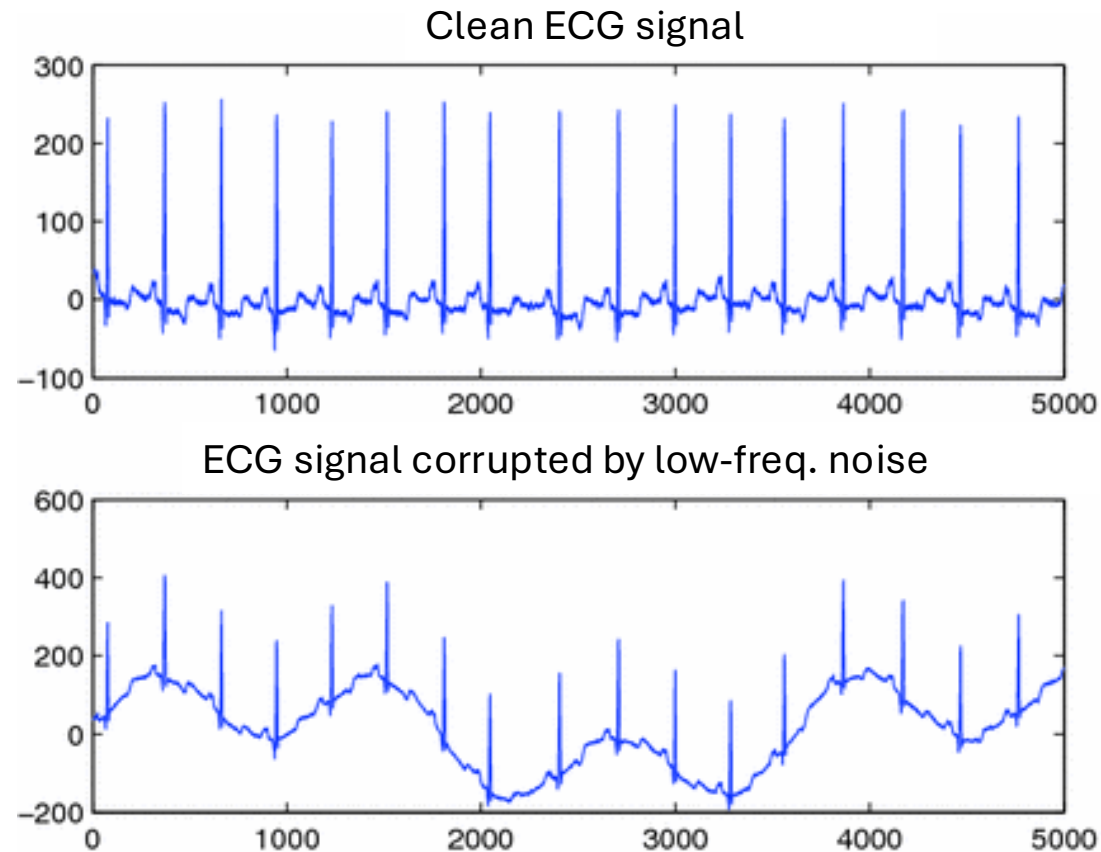
Denoising

- Raw signals are often noisy: the signal of interest is corrupted by noise
- Example: high-frequency noise on a CGM signal



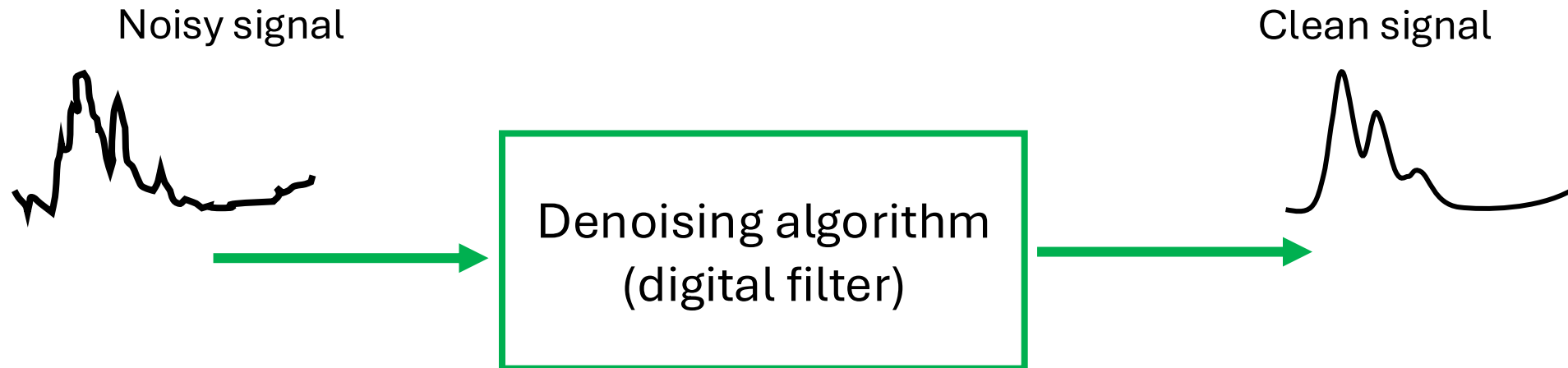
Denoising

- Raw signals are often noisy: the signal of interest is corrupted by noise
- Example: low-frequency noise (e.g., due to movement) on an ECG signal



Denoising

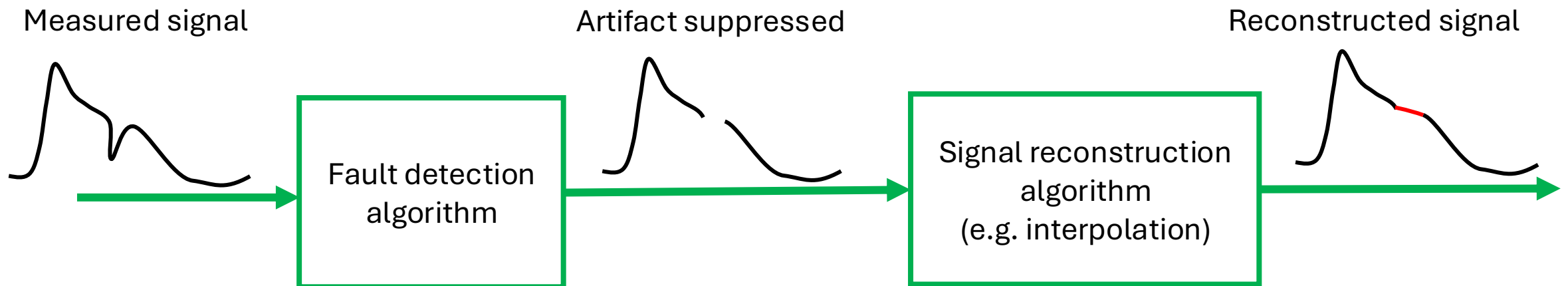
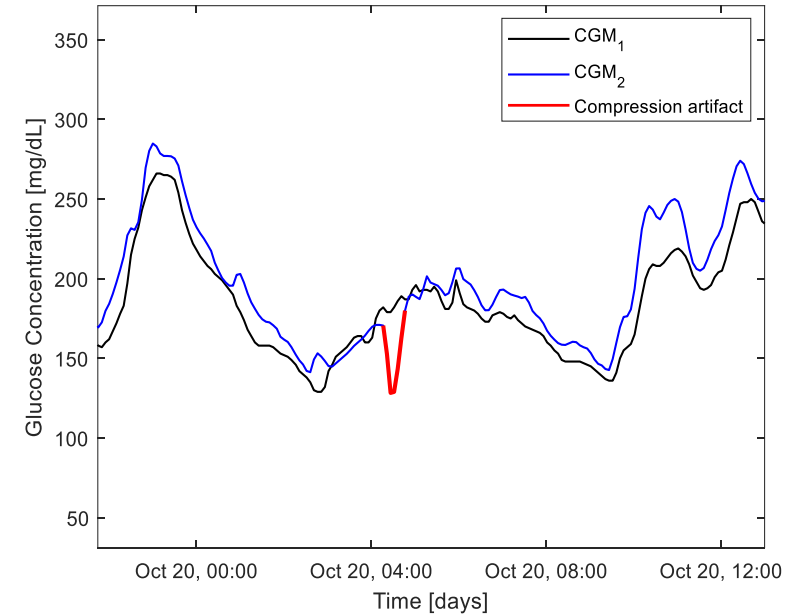
- The noise can be filtered by applying a denoising algorithm exploiting a digital filter.



- Non-causal filters: the filter output depends on past and future inputs → for offline use only
- Causal filters: the filter output depends only on past and present inputs → suitable for online application

Fault detection

- Detection of temporary sensor errors also called faults or artifacts
- Example: pressure-induced signal suppression in a CGM signal



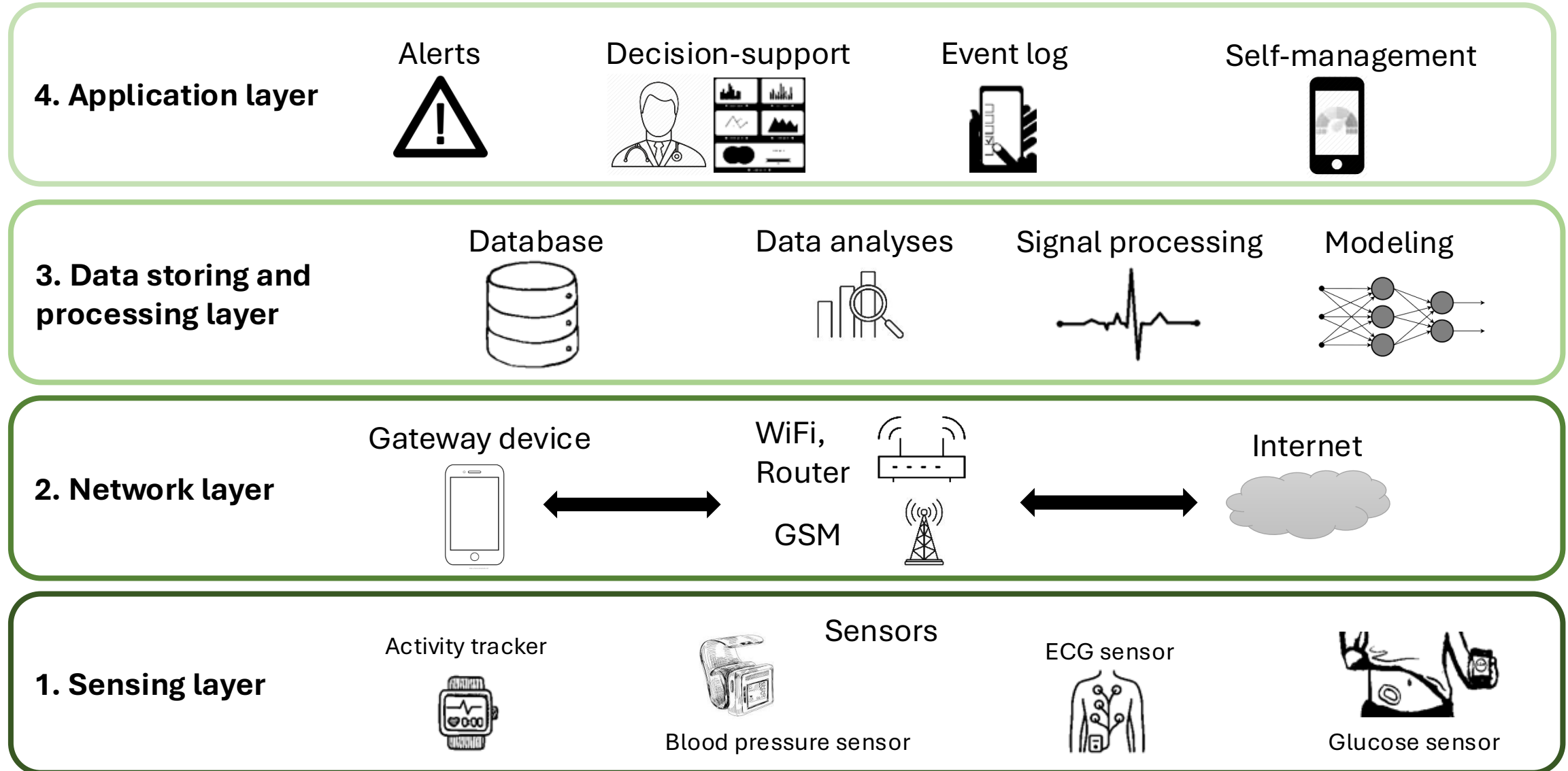
Communication systems

BONUS

- Wearable devices require a data transmission systems that **wirelessly** communicate between the sensing device and the receiver and/or gateway device (it can be the smartphone).
- **Bluetooth communication** is the primary choice because of its simplicity, low operation cost, compatibility with new smartphone technology, and simple hardware. Drawback: it requires high power consumption.
- **Near field communication (NFC):** a wireless communication technology that operates at a short distance (about 4 cm) and does not need the energy from the battery to transfer the data.
- NFC tags can directly communicate with the smartphone equipped with the NFC modules.
- Bypassing the need for batteries, NFC enables ultrathin, ultraminiaturized designs for sensors, like a temporary transfer tattoo.



The architecture of wearable IoT systems



1. Sensing layer

- This layer includes sensors, which are responsible for:
 - **Collecting the data** either directly from the body through contact sensors or from peripheral sensors providing indirect information of body and behaviours
 - **Preparing the data** for on-board analysis, closed-loop feedback or remote transmission for more comprehensive analysis and storing
- Key components of IoT wearable sensors:
 - Miniaturized **sensor hardware**
 - **Embedded processor** with storage capabilities, power management and communication circuits
- Typically, limited computing power and storage capabilities.

Contact sensors

ECG sensor



Glucose sensor



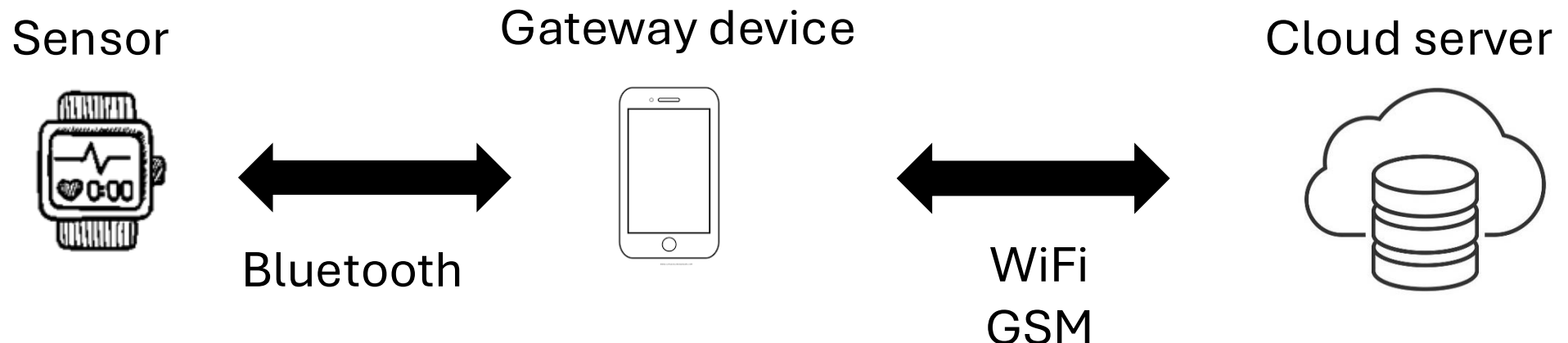
Peripheral sensors

Step counter Microphone and camera



2. Network layer

- For more comprehensive data analysis and storage, the sensor layer needs to transmit the data to **cloud computing servers** (remotely-located computing machines).
- This transmission can be mediated by a companion device that serves as a **gateway**, i.e., it enables the information to flow from sensors to the cloud server.
 - Example of gateway devices: smartphones, tablets and PCs.
- The communication must be **secured** through:
 - Authentication protocols for writing, modifying and accessing the data
 - Security and privacy protocols for preserving the data



3. Data storing and processing layer

- Layer responsible for data storing and complex processing

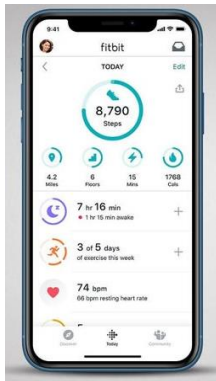
- Key components:
 - **Database** for structured data storing
 - Data organized based on a specific data model (relational, hierarchical, object-oriented,...)

 - **Algorithms** for data processing
 - Statistical analysis
 - Feature extraction (e.g., heart rate, calories expenditure, ...)
 - Signal processing (e.g., filtering, artifact removal, ...)
 - Data mining (e.g., cluster analysis, anomaly detection, sequential pattern extraction, ...)
 - Data modeling (knowledge-based, data-driven or hybrid approaches)
 - Artificial intelligence (knowledge representation, machine learning, natural language processing, machine perception, automated planning, ...)

4. Application layer

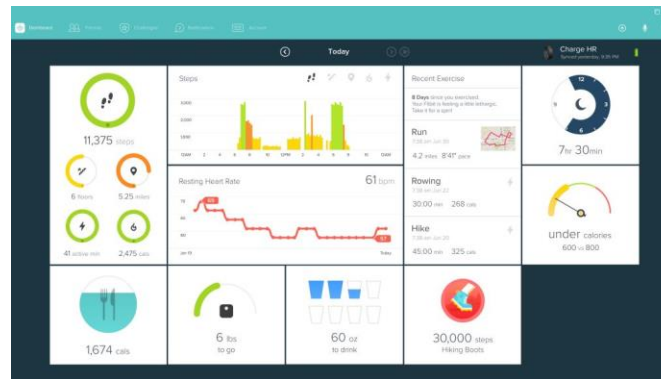
- Layer responsible for visualizing raw or processed data to end-users and for providing decision support to them through a software application
- Possible end-users: patients, physicians, caregivers, analysts, data managers, administrators, etc.
- Possible types of applications:

Mobile app



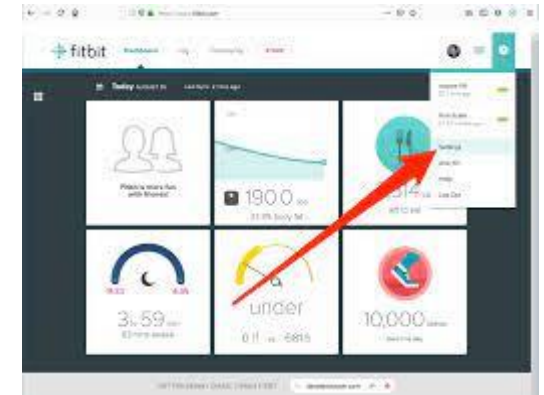
App installed in a mobile device (e.g. smartphone, smartwatch, tablet, etc.)

Desktop app



App installed in a computer. It can work offline.

Web app



App that runs on the browser. Installation not required. Internet access required.

Appendix: Data source characteristics

Focus on data – Calories

➤ Calories: a list with 1 data point exactly every minute

➤ Fields meaning:

- time: timestamp (hh:mm:ss format) of the calory entry
- value: the calories spent during the last 1 minute (rest + active)

```
[  
  {  
    "time": "00:00:00",  
    "value": "1.29"  
  },  
  {  
    "time": "00:01:00",  
    "value": "1.29"  
  },  
  {  
    "time": "00:02:00",  
    "value": "1.29"  
  },  
  {  
    "time": "00:03:00",  
    "value": "1.55"  
  },  
  {  
    "time": "00:04:00",  
    "value": "1.29"  
  },  
  {  
    "time": "00:05:00",  
    "value": "1.29"  
  },  
  {  
    "time": "00:06:00",  
    "value": "1.29"  
  },  
  ...  
]
```

Focus on data – Distance

- Distance: a list with 1 data point every minute or more depending on user's movement
- Fields meaning:
 - time: timestamp (hh:mm:ss format) of the distance entry
 - value: the distance travelled between the last timestamp and now (in cm)

```
[
  {
    "time": "00:03:00",
    "value": "0"
  },
  {
    "time": "00:50:00",
    "value": "0"
  },
  {
    "time": "01:28:00",
    "value": "0"
  },
  {
    "time": "01:32:00",
    "value": "1440"
  },
  {
    "time": "01:33:00",
    "value": "3110"
  },
  {
    "time": "01:34:00",
    "value": "0"
  },
  {
    "time": "01:53:00",
    "value": "0"
  },
  {
    "time": "03:09:00",
    "value": "0"
  },
  ...
]
```


Focus on data – Steps

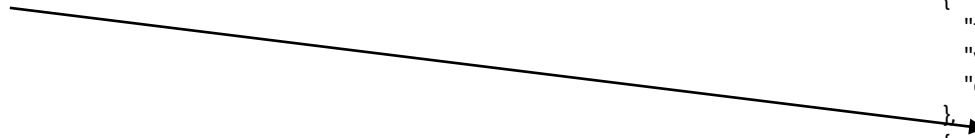
- Steps: a list with 1 data point every minute or more depending on user's movement
- Fields meaning:
 - time: timestamp (hh:mm:ss format) of the step entry
 - value: the number of steps done between the last timestamp and now
- **Note:** there can be days without data

```
[
  {
    "time": "00:07:00",
    "value": "0"
  },
  {
    "time": "00:10:00",
    "value": "0"
  },
  {
    "time": "01:57:00",
    "value": "0"
  },
  {
    "time": "02:29:00",
    "value": "0"
  },
  {
    "time": "02:40:00",
    "value": "0"
  },
  {
    "time": "02:57:00",
    "value": "11"
  },
  {
    "time": "02:58:00",
    "value": "18"
  },
  {
    "time": "02:59:00",
    "value": "0"
  },
  ...
]
```

Heart rate

- Heart rate: a list with 1 data point every 5 seconds.
- Fields meaning:
 - time: timestamp (hh:mm:ss format) of the heart rate entry
 - value: the heart rate at the current time (in bpm)
 - confidence: a number from 0 to 3 defining how much the reading is reliable (this probably depends on user movement, etc. You can ignore it)
- **Note:** there can be missing values

```
[  
  {  
    "time": "03:10:19",  
    "value": 58,  
    "confidence": 1  
  },  
  {  
    "time": "03:10:24",  
    "value": 54,  
    "confidence": 3  
  },  
  {  
    "time": "03:10:29",  
    "value": 52,  
    "confidence": 3  
  },  
  {  
    "time": "03:10:44",  
    "value": 52,  
    "confidence": 3  
  },  
  {  
    "time": "03:10:54",  
    "value": 53,  
    "confidence": 3  
  },  
  {  
    "time": "03:11:09",  
    "value": 54,  
    "confidence": 3  
  },  
  {  
    "time": "03:11:19",  
    "value": 53,  
    "confidence": 3  
  },  
  ...  
]
```



Resting heart rate

- Resting heart rate: an entry per day.

```
{  
  "time": "00:00:00",  
  "value": 51.35,  
  "error": 17.86  
}
```

- Fields meaning:

- time: timestamp (hh:mm:ss format) of the resting heart rate entry (always 00:00:00)
- value: the estimated resting heart rate at the current time (in bpm)
- error: the estimate error (in %) (Can be ignored)

- **Note:** there can be days without data

Exercise

- Exercise: a list with 1 entry for each recorded exercise session
- Fields meaning:
 - activityName: the type of entry (“Corsa”: Run, “Bici”: bike, “Camminata”: Walk)
 - averageHeartRate: the average heart rate during the session
 - calories: the active calories spent during the session
 - distance: the distance covered during the session
 - distanceUnit: the unit of the “distance” field
 - duration: the total duration of the session (in ms)
 - activeDuration: the total active duration of the session (in ms)
 - steps: the total number of steps done during the session (if “Corsa” or “Camminata”)
 - logType: how this record was logged (“tracker” or “auto_detected”)
 - heartRateZones: a list that summarizes the number of minutes and calories spent in specific heart rate ranges (from min to max)
 - speed: the average speed (in km/h)
 - vo2max: the estimated running VO2Max (only for “Corsa” activities)
 - elevationGain: the elevation gained during the session
 - time: the starting time of the session (format hh:mm:ss)
- **Note:** different activity types can have (slightly) different fields
- **Note:** there can be days without data

```
[
  {
    "activityName": "Corsa",
    "averageHeartRate": 143,
    "calories": 727,
    "distance": 8.474117,
    "distanceUnit": "Kilometer",
    "duration": 3.548E+6,
    "activeDuration": 3.214E+6,
    "steps": 8604,
    "logType": "tracker",
    "heartRateZones": [
      {
        "name": "Fuori zona",
        "min": 30,
        "max": 110,
        "minutes": 3,
        "caloriesOut": 30.146219999999971
      },
      {
        "name": "Grassi bruciati",
        "min": 110,
        "max": 137,
        "minutes": 8,
        "caloriesOut": 97.13782
      },
      {
        "name": "Attivita aerobica",
        "min": 137,
        "max": 170,
        "minutes": 48,
        "caloriesOut": 659.73843
      }
    ],
    "speed": 9.4981754822650917,
    "vo2Max": {
      "vo2Max": 48.77853
    },
    "elevationGain": 28.042,
    "time": "11:22:43"
  },
  {
    "activityName": "Bici",
    "averageHeartRate": 150,
    "calories": 3893,
    "distance": 140.143695,
    "distanceUnit": "Kilometer",
    "duration": 2.0901E+7,
    "activeDuration": 1.756E+7,
    "logType": "tracker",
    "heartRateZones": [
      {
        "name": "Fuori zona",
        "min": 30,
        "max": 110,
        "minutes": 28,
        "caloriesOut": 214.835279999999842
      },
      {
        "name": "Grassi bruciati",
        "min": 110,
        "max": 136,
        "minutes": 64,
        "caloriesOut": 664.550279999999993
      },
      {
        "name": "Attivita aerobica",
        "min": 136,
        "max": 170,
        "minutes": 232,
        "caloriesOut": 3167.1500100000012
      },
      {
        "name": "Picco",
        "min": 170,
        "max": 220,
        "minutes": 24,
        "caloriesOut": 377.760599999999995
      }
    ],
    "speed": 28.734978587699313,
    "elevationGain": 663.55,
    "time": "11:03:54"
  },
  ...
]
```

Sleep

➤ Sleep: a list with 1 entry for each sleep session

➤ Fields meaning:

- dateOfSleep: day associated to the sleep entry (MM-DD format)
- startTime: the starting timestamp of the sleep (MM-DD hh:mm:ss format)
- endTime: the ending timestamp of the sleep (MM-DD hh:mm:ss format)
- duration: the duration of the sleep session (in ms)
- minutesToFallAsleep: the number of minutes spent to fall asleep
- minutesAsleep: the number of minutes asleep during the sleep entry
- minutesAwake: the number of minutes awake during the sleep entry
- minutesAfterWakeup: the number of minutes the user spent in bed after waking up
- efficiency: the estimated sleep efficiency (from 0 to 100)
- logType: how the entry was logged (always “auto_detected”)
- mainSleep: a boolean indicating if the entry corresponds to the main sleep session
- levels: a summary of the sleeping stages (deep/wake/light/rem/restless) during the sleep session
- data: the stage profile of the sleep session. Each entry contains the starting timestamp, the level and how much it lasted.

➤ **Note:** there can be days without data

```
{
  {
    "dateOfSleep": "02-14",
    "startTime": "02-13 22:44:00",
    "endTime": "02-14 06:36:30",
    "duration": 2.832E+7,
    "minutesToFallAsleep": 0,
    "minutesAsleep": 429,
    "minutesAwake": 43,
    "minutesAfterWakeup": 3,
    "timeInBed": 472,
    "efficiency": 96,
    "logType": "auto_detected",
    "mainSleep": true,
    "levels": {
      "summary": {
        "deep": {
          "count": 5,
          "minutes": 106,
          "thirtyDayAvgMinutes": 86
        },
        "wake": {
          "count": 34,
          "minutes": 43,
          "thirtyDayAvgMinutes": 46
        },
        "light": {
          "count": 28,
          "minutes": 235,
          "thirtyDayAvgMinutes": 281
        },
        "rem": {
          "count": 11,
          "minutes": 88,
          "thirtyDayAvgMinutes": 101
        }
      },
      "data": [
        {
          "dateTime": "02-13 22:44:00",
          "level": "wake",
          "seconds": 30
        },
        {
          "dateTime": "02-13 22:44:30",
          "level": "light",
          "seconds": 150
        },
        {
          "dateTime": "02-13 22:47:00",
          "level": "deep",
          "seconds": 1860
        },
        ...
      ]
    }
  },
  ...
]
```